

Universidad Tecnológica Nacional

**Tecnicatura Universitaria en Programación a
distancia**

Programación I

**Trabajo Práctico Integrador
Gestión de Datos de Países en Python: filtros,
ordenamientos y estadísticas**

Alumnos

Rodríguez Carolina

Reyes Lima Juan Manuel

Comisión 27

Fecha de entrega: 14/06/2026

Índice

1. Introducción.....	3
2. Marco teórico.....	3
2.1. Herramientas de control de flujo.....	3
2.1.1 Estructuras condicionales if / elif / else.....	3
2.1.2. Bucles: for y while.....	4
2.1.3. Bloques de control try / except / else.....	4
2. 2. Funciones.....	5
2.2.1. Parámetros y argumentos.....	5
2.3. Listas.....	6
2.4 Dicionarios.....	7
2.5. Archivos.....	7
2.5.1 Apertura de archivos.....	7
2.5.2. Archivos CSV.....	8
2.6. Ordenamiento.....	9
2.7. Estadísticas básicas.....	9
3. Decisiones técnicas y Arquitectura.....	10
3.1. Diagrama de flujo.....	10
3.2. Capturas de pantalla de la ejecución de ejemplos.....	11
4. Dificultades y conclusiones.....	14
4.1. Dificultades a la hora de resolver el programa.....	14
4.2. Conclusión final.....	14
5. Bibliografía y links.....	15

1. Introducción

Para el presente trabajo se desarrolló una aplicación en Python que permite gestionar información sobre países, aplicando listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos y estadísticas. El sistema lee datos desde un archivo CSV, realiza consultas y genera indicadores clave a partir del dataset. A continuación se explicarán los conceptos teóricos aplicados para este trabajo.

2. Marco teórico

El presente marco teórico tiene como finalidad explicar los conceptos fundamentales del lenguaje de programación Python aplicados para la resolución del Trabajo Integrador y su importancia para comprender las decisiones de diseño tomadas para su resolución.

2.1. Herramientas de control de flujo

Las herramientas de control de flujo son muy importantes para controlar o dirigir el flujo de ejecución del programa. De lo contrario, el programa se ejecutaría de forma lineal, sin la posibilidad de tomar decisiones o repetir acciones.

Estas herramientas o sentencias de control de flujo evalúan una o varias **condiciones** y a partir de su resultado de verdad (resultado booleano: Verdadero o Falso) ejecutarán o no un bloque de instrucciones.

2.1.1 Estructuras condicionales *if* / *elif* / *else*

Los ***if*** son el tipo más común de sentencias de control de flujo, donde el bloque contenido dentro de esta sentencia se ejecutará una sola vez, sólo si la condición es Verdadera.

La sentencia ***else*** es opcional y puede seguir a una sentencia *if*, donde sólo se ejecutará si la sentencia *if* es Falsa. La sentencia *else* no tiene una condición para ejecutar su bloque de código, o más bien, su condición es que la sentencia *if* sea Falsa.

La sentencia ***elif*** se utiliza cuando existen varias alternativas o cláusulas posibles. Es una sentencia que siempre sigue de un *if* u otro *elif*. Proporciona otra condición que sólo se comprueba si las anteriores (*if* o *elif*) fueron Falsas.

Para definir estos condicionales en Python, se usa:

- Palabra clave ***if* / *elif* / *else*** seguido de la condición o expresión que se evalúa como Verdadero o Falso (excepto en el *else* que no tiene condición y solo se ejecuta si el *if* y los *elif*, si los hubiera, resultan Falsos)
- dos puntos (:)
- y el bloque de código que se quiere ejecutar si la condición es verdadera.

```
num = 10
if (num > 2):
    print(f"{num} es mayor a 2")
elif (num == 2):
```

```
print(f"{num} es igual a 2")
else:
    print(f"{num} es menor a 2")
```

2.1.2. Bucles: for y while

Los bucles se utilizan para repetir un bloque de código una cantidad determinada o indefinida de veces.

La **sentencia for** ejecutará su bloque de código una cantidad determinada de veces. No tendrá una condición de Verdadero o Falso, sino que en su condición estarán la cantidad de veces que debe ejecutarse el bucle.

La sentencia más típica es: **for i in range(5)** donde:

- **for** es la palabra clave
- **i** es la variable que itinerará por los valores
- la palabra clave **in**
- la llamada a la función **range()** que devuelve la cantidad de iteraciones que hará **i**
- y el bloque de código a ejecutar

En nuestro programa, se suele utilizar la sentencia **for** para recorrer los elementos o claves de listas o diccionarios:

```
for diccionario in lector_dict:
    # Bloque de código
```

En este caso, **lector_dict** contiene un iterador que sabe cómo leer un archivo .csv fila por fila y convertir cada una en un diccionario. Y **diccionario** es un diccionario que representa una fila del archivo .csv, donde las **claves** son los encabezados de las columnas y los **valores** son los datos de las filas.

La **sentencia while** ejecutará su bloque de código de manera constante mientras su condición sea verdadera. Para definir un while en Python:

- Palabra clave **while**
- una condición que se evalúa como Verdadera o Falsa
- seguido de dos puntos (:)
- y el bloque de código a ejecutar

2.1.3. Bloques de control try / except / else

Son estructuras que permiten manejar errores y controlar lo que sucede cuando ocurre un error (excepción) durante la ejecución del programa. Son una forma de evitar que el programa falle y se cierre por un ingreso de dato no esperado, de manera que se pueda controlar y gestionar esos errores para que el programa siga funcionando.

El bloque **try** contendrá código que puede llegar a fallar, donde habrá manipulaciones de datos o conversiones que pueden hacer que el programa falle y se cierre inesperadamente.

El bloque **except** contiene código que se ejecuta sólo si ocurre un error en el bloque **try**, sirviendo como una escapatoria para que el programa no falle y se cierre.

El bloque **else** en este complejo **try/except/else** sólo se ejecuta si nada falla en el bloque **try**.

En nuestro programa, se utiliza mucho la sentencia **while True** en funciones de validación de datos junto con bloques **try / except / else**.

```
def validar_dato(dato):  
    while True:  
        try:  
            #bloque try que verifica posibles errores  
        except:  
            #bloque except que informa errores  
            #y vuelve a solicitar el dato  
        else:  
            return dato
```

En este ejemplo, el bloque **try** verifica que el dato sea el esperado (por ejemplo, que no sea un campo vacío, que sea un dato numérico, etc.). Si el dato no es el esperado, se ejecutará el bloque **except** informando el error y volviendo a solicitar el dato. El bloque **else** sólo se ejecutará si en el bloque **try** no se produce ningún error. Por lo tanto, este bloque **while** se ejecutará hasta que el programa ingrese al bloque **else** y devuelva un valor de retorno.

2. 2. Funciones

Las funciones son bloques de código reutilizables que realizan una tarea específica. Se definen una sola vez al inicio del programa, antes del bloque de código principal, y pueden ser llamadas todas las veces que sea necesario. Por lo que evitan tener que escribir las mismas líneas de código varias veces.

En Python, se definen:

- Palabra clave **def**
- nombre representativo de la función
- paréntesis, en donde se les puede pasar parámetros opcionales
- dos puntos (:)
- bloque de código a ejecutar
- **return** que es el valor que devuelve la función (opcional)

2.2.1. Parámetros y argumentos

Un **parámetro** es la variable que se define en la función como espacio que espera recibir un valor. Y un **argumento** es el valor concreto que se le pasa a la función cuando se la llama:

```
def saludar(nombre): # 'nombre' es el parámetro  
    print("Hola,", nombre)  
saludar("Ana") # "Ana" es el argumento
```

En Python se pueden usar **parámetros con valor por defecto**. Donde se les asigna directamente un valor en la definición de la función. Cuando se define un parámetro con valor por defecto, se puede llamar a la función sin ese argumento, y la función usará el valor por defecto. O se puede llamar a la función ingresándole ese argumento, y la función usará el valor que se le pase.

En nuestro programa se utiliza esto para el desarrollo de la Opción 5: Ordenar países. En esta opción, el programa permite ordenar los países según tres criterios: por nombre, por población o por superficie (ascendente o descendente). Para resolver los 3 puntos, se utiliza la misma función, a la cuál se le pasa como argumentos el **lector** que es el iterador del archivo .csv, el **criterio** que corresponde a una cadena de caracteres que define el criterio de ordenamiento ("nombre", "población", "superficie"), y un parámetro con valor por defecto llamado **ascendente** al que se le asigna el valor False.

Esto permite utilizar ese valor en la función **sorted()** la cual ordena los valores de menor a mayor por defecto. La función **sorted()** tiene la posibilidad de pasarle un valor al parámetro **reverse=** para invertir el orden de ordenamiento de mayor a menor. De esta manera, el parámetro **ascendente** determina el valor que se le pasa a **reverse=**, permitiendo controlar el orden desde la llamada a la función.

2.3. Listas

Una lista constituye una estructura de datos que permite almacenar una secuencia ordenada de elementos, los cuales pueden corresponder a cualquier tipo de dato, incluyendo diccionarios (aspecto de particular relevancia para el presente trabajo). Las listas se caracterizan por ser mutables y se definen mediante el uso de corchetes, separando sus elementos por comas. El acceso a los distintos elementos se realiza a través de índices, lo cual confiere a las listas la propiedad de ser objetos iterables.

En el lenguaje Python, las listas resultan de particular interés debido a la diversidad de métodos que pueden aplicarse sobre ellas. Un método puede definirse como una operación predefinida que se ejecuta sobre un objeto, y cuya sintaxis consiste en escribir el nombre del objeto, seguido de un punto y, posteriormente, el nombre del método correspondiente. Estas operaciones se encuentran predefinidas en la biblioteca estándar de Python.

En el desarrollo de nuestro programa, se empleó como estructura auxiliar una lista de diccionarios para la implementación de determinadas funciones (filtrado de países, actualización de archivos y búsqueda de países por nombre), para lo cual se utilizó el método **.append()**. Dicho método permite incorporar, al final de la lista, los datos recibidos como parámetro. En los tres casos mencionados, los datos almacenados correspondieron a diccionarios que posteriormente serían mostrados en pantalla o escritos en un archivo, según el requerimiento específico de cada función.

La elección de esta estructura particular responde a la conveniencia que ofrece para aprovechar los formatos de lectura y escritura de archivos .CSV basados en diccionarios.

```
aux_list = []
lector_dict = csv.DictReader(ar)
for diccionario in lector_dict:
    if texto == diccionario['nombre'] or texto in diccionario['nombre']:
        aux_list.append(diccionario)
```

2.4 Diccionarios

Un diccionario constituye una estructura de datos que permite almacenar información mediante un sistema de pares clave-valor, donde cada elemento se compone de una clave (key) única, asociada a un valor (value) determinado. Dichos valores pueden corresponder a cualquier tipo de dato, y los diccionarios se caracterizan por ser mutables.

En el desarrollo del presente programa, cada línea del archivo "datos_países.CSV" es leída y registrada en forma de diccionario, en el cual las claves corresponden a los campos del encabezado del archivo (nombre, población, superficie, continente). En aquellos casos en que resulta necesario, estos diccionarios se incorporan, a su vez, a una lista auxiliar, conforme a lo expuesto en la sección anterior.

El empleo de diccionarios resultó conveniente para el presente trabajo, dado que facilita tanto el manejo de los datos como la manipulación del archivo .CSV a través de funciones como **DictReader()** y **DictWriter()**. Estas funciones reconocen de manera automática el encabezado del archivo y lo emplean como conjunto de claves de los diccionarios en los que se almacena la información.

```
with open(archivo, "r", encoding="utf-8", newline="") as ar:
    lector_dict = csv.DictReader(ar)
    for diccionario in lector_dict:
        if texto == diccionario['nombre']:
            if submenu == 1:
                diccionario["poblacion"] = numero
            elif submenu == 2:
                diccionario["superficie"] = numero
```

2.5. Archivos

Un archivo son datos que se almacenan en el disco, y pueden ser letras, números, imágenes, etc. Gracias a los archivos se pueden guardar datos de manera persistente que de lo contrario se perderían al finalizar el programa.

2.5.1 Apertura de archivos

Para abrir un archivo se utiliza la sentencia:

```
with open("archivo.csv", "modo", encoding="utf-8") as archivo:
```

with open es una forma segura de trabajar con archivos ya que abre el archivo y lo cierra automáticamente al finalizar el bloque de ejecución. Es más seguro ya que si ocurriese un error mientras se trabaja con el archivo, igual lo cerrará automáticamente.

“**modo**” corresponde al modo de apertura del archivo, el cual puede ser:

- “r” para lectura del contenido, sin posibilidad de modificarlo.
- “w” para escritura que crea un archivo o sobrescribe el existente.
- “a” para agregar contenido al final del archivo sin borrar lo existente.

Existen otros modos de apertura, pero estos son los que utilizamos en nuestro programa.

encoding=“utf-8” es un parámetro opcional que especifica la codificación de caracteres con la que se debe leer o escribir el archivo. Si esto no se especifica, Python usa la codificación por defecto del sistema operativo, que puede causar errores o problemas al visualizar caracteres con tilde, por ejemplo.

UTF-8 es una codificación que soporta caracteres con tilde, las eñes (ñ) y caracteres especiales, como por ejemplo ‘ü’.

En nuestro programa, dado que almacenamos datos de países como nombre y continente, es importante poder agregarles los tildes u otros caracteres sin errores o textos corruptos.

2.5.2. Archivos CSV

En nuestro programa trabajamos con archivos con extensión **.csv**.

CSV (Comma Separated Values o Valores Separados por Coma) es un tipo de archivo de texto plano que almacena datos en forma de tabla. Se puede abrir desde un editor de texto donde se verá simplemente el texto plano, pero tiene una estructura interna similar a una hoja de cálculo, en donde cada línea del texto representa una **fila**, y cada elemento separado por coma (,) o punto y coma (;) representa una **celda** individual.

En la primera línea de los archivos CSV suele ir un encabezado, que corresponden a los nombres de las **columnas**. Por lo que un archivo CSV se asemeja a una representación de filas, columnas y celdas.

En nuestro programa, el archivo CSV funciona como base de datos donde se almacena la información de los países. Contiene los campos *nombre*, *población*, *superficie* y *continente*, que representan las columnas y los atributos de cada país registrado.

Para trabajar con este tipo de archivos, Python posee el módulo csv dentro de su librería estándar. En particular, utilizamos **csv.DictWriter** y **csv.DictReader** para leer y escribir el archivo fila por fila convirtiendo cada registro en un diccionario, donde las **claves** serán los encabezados del archivo, y los **valores** serán los datos de cada país. Esto facilita el acceso a los datos por el nombre de la columna (la clave en el diccionario).

csv.DictWriter se usa en funciones de agregar un nuevo país con todos sus datos o actualizar la población o superficie de un país existente. **csv.DictReader** se utiliza en funciones de búsqueda, filtrado, ordenamiento y cálculo de estadísticas.

2.6. Ordenamiento

El ordenamiento es el proceso de organizar los datos según un criterio. En Python, una de las herramientas principales que utilizamos para esto es la función **sorted()** que recibe un iterable y devuelve una nueva lista con los elementos ordenados, sin modificar la estructura principal.

A la función **sorted()** se le puede pasar como parámetro **key=**, que permite definir un criterio de ordenamiento, y el parámetro **reverse=** que invierte el orden cuando se le asigna el valor True (por defecto, el orden será de menor a mayor)

En nuestro programa, se utiliza una sola función para ordenar los países según tres criterios: nombre, población y superficie (ascendente o descendente). Para definir el criterio se utiliza la función **lambda**, que indica que el criterio de ordenamiento es el valor de la clave “nombre”, “población” o “superficie”.

Por ejemplo:

```
sorted(lector, key= lambda clave: int(clave["poblacion"]))
```

lambda indica que el criterio de ordenamiento es la clave “población” pero teniendo en cuenta sus valores como **enteros**.

2.7. Estadísticas básicas

Las estadísticas son cálculos que resumen o describen a un conjunto de datos. En nuestro programa se calculan las siguientes estadísticas:

- **País con mayor y menor población:** se calcula creando una lista auxiliar ordenada (función **sorted()**) por el criterio de “población”, para luego informar el primero y el último valor que corresponden al país con menor y mayor población respectivamente.
- **Promedio de población y superficie:** para estos puntos se recorre todo el archivo CSV a través de un bucle **for** que recorre cada fila del archivo tratada como una lista de diccionarios. Se hará la sumatoria de los valores correspondiente a la clave “población” o “superficie” y se dividirá por la cantidad total de países para calcular el promedio.
- **Cantidad de países por continente:** se calcula creando un diccionario auxiliar en donde se guardará como **clave** el nombre del continente y como **valor** la cantidad de países que pertenezcan a ese continente.

3. Decisiones técnicas y Arquitectura

3.1. Diagrama de flujo

A continuación se muestra el diagrama de flujo de las operaciones principales del programa.

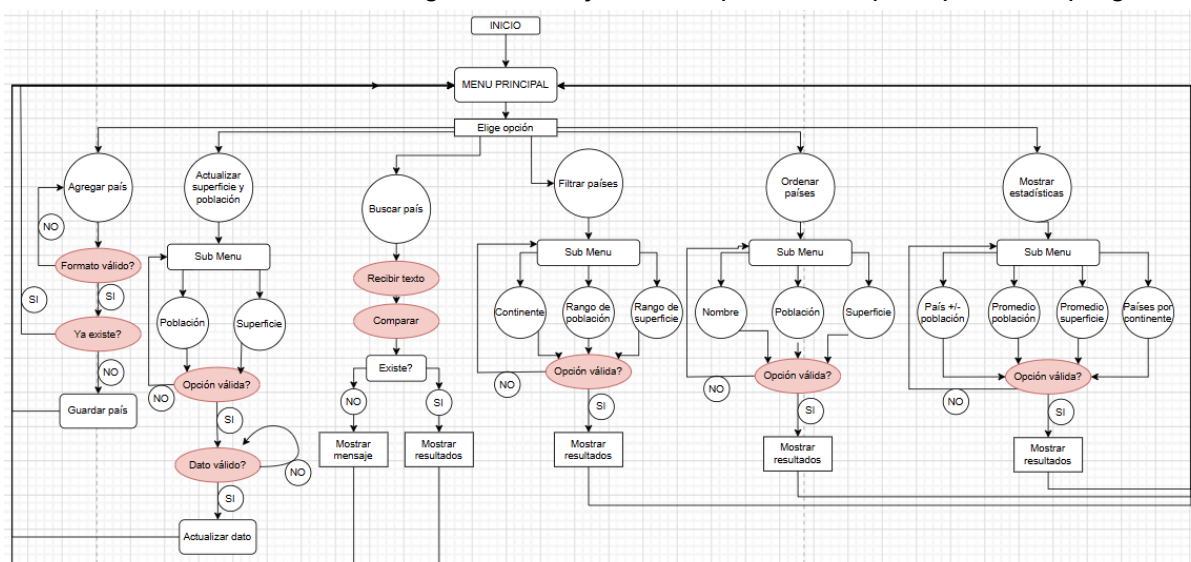


Imagen 1: Diagrama de flujo completo

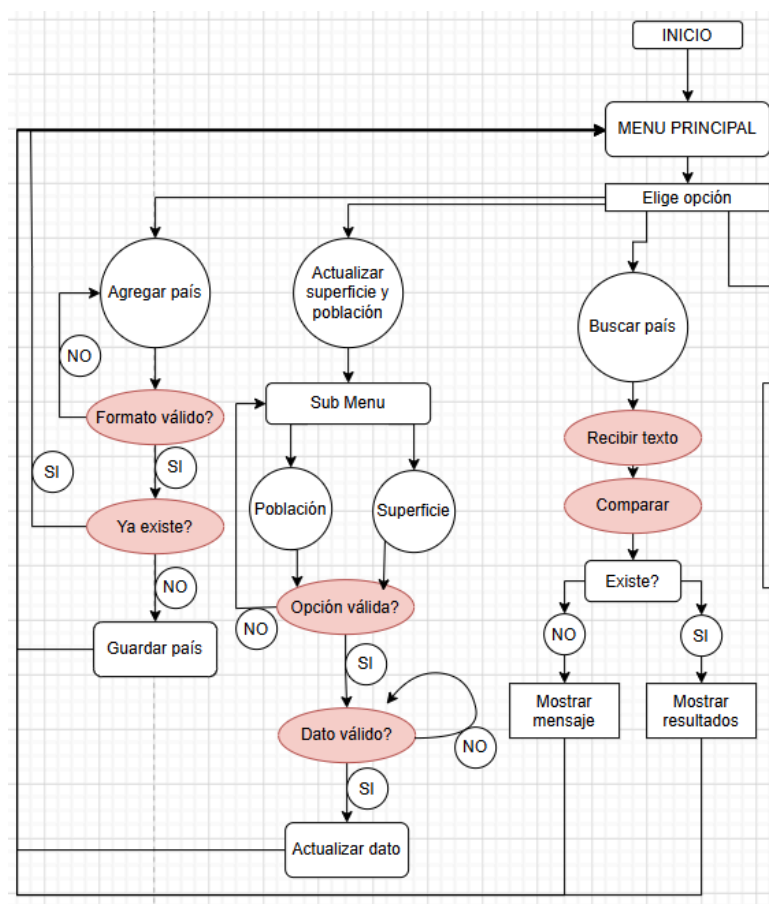


Imagen 2: Diagrama de flujo ampliado. Opciones 1, 2 y 3

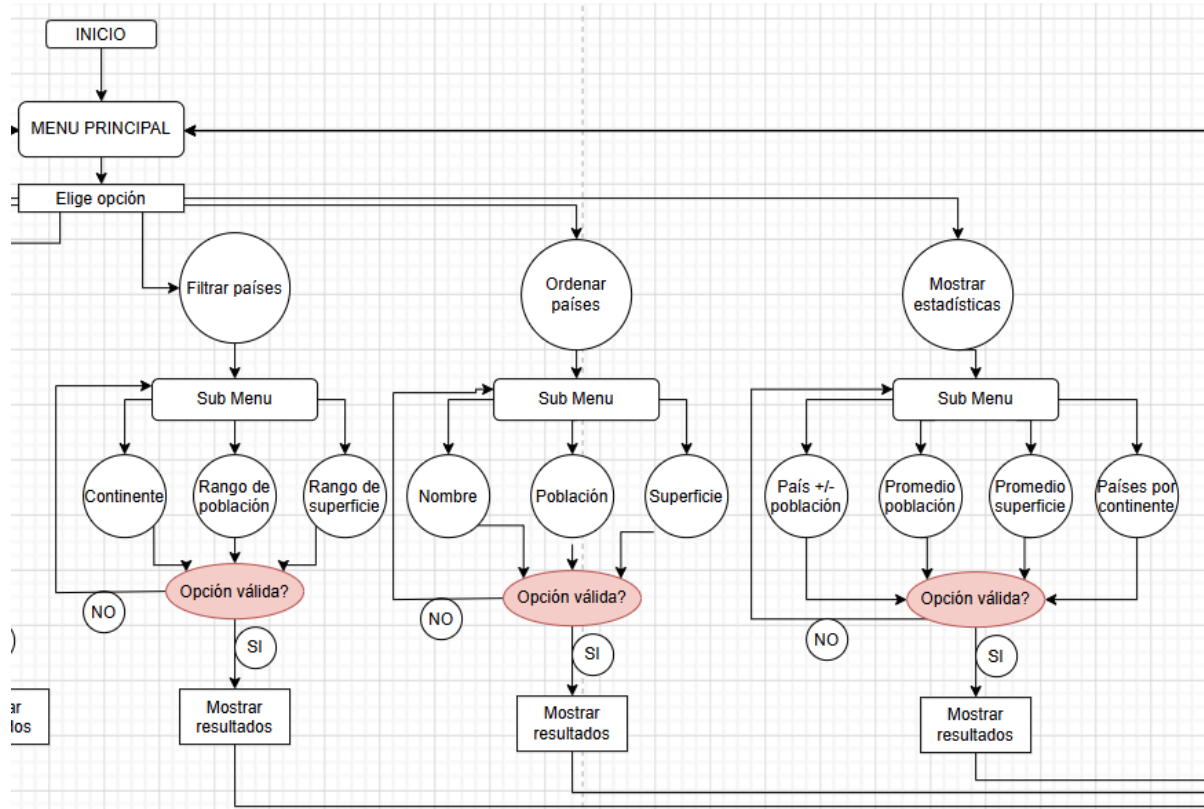


Imagen 3: Diagrama de flujo ampliado. Opciones 4, 5 y 6

3.2. Capturas de pantalla de la ejecución de ejemplos

```

#####
Bienvenido al gestor de información de países
#####
Elija una opción del menú:
1) Agregar un país
2) Actualizar los datos de Población y Superficie
3) Buscar un país por nombre
4) Filtrar países (submenú)
5) Ordenar países (submenú)
6) Mostrar Estadísticas (submenú)
7) Salir
Ingrese una opción: 5
Ordenar países por:
1) Nombre
2) Población
3) Superficie (ascendente o descendente)
Ingrese la opción: 2

Países ordenados por poblacion:
Nombre | Población | Superficie | Continete
Suiza | 9048905 | 41291km2 | Europa
Argentina | 46735004 | 2780400km2 | América
Japón | 123719238 | 377975km2 | Asia
Brasil | 213562666 | 8515770km2 | América
Nigeria | 242431832 | 923768km2 | África
Presione Enter para continuar...
  
```

Imagen 4: Ejecución de Opción 5.2: Ordenar países por población.

```
#####
Bienvenido al gestor de información de países
#####
Elija una opción del menú:
1) Agregar un país
2) Actualizar los datos de Población y Superficie
3) Buscar un país por nombre
4) Filtrar países (submenú)
5) Ordenar países (submenú)
6) Mostrar Estadísticas (submenú)
7) Salir
Ingrese una opción: 6

Mostrar estadísticas de:
1) País con mayor y menor población
2) Promedio de población
3) Promedio de superficie
4) Cantidad de países por continente
Ingrese la opción: 1

País con menor población: Suiza con 9048905 habitantes.
País con mayor población: Nigeria con 242431832 habitantes.
Presione Enter para continuar...|
```

Imagen 5: Ejecución de opción 6.1: Mostrar estadística de país con mayor y menor población

```
#####
Bienvenido al gestor de información de países
#####
Elija una opción del menú:
1) Agregar un país
2) Actualizar los datos
3) Buscar un país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar Estadísticas
7) Salir
Ingrese una opción: 1
Ingrese el nombre del país que desea agregar: argelia
Ingrese la población de Argelia: 46810000
Ingrese la superficie de Argelia [km**2]: 2381740
Ingrese el continente al que pertenece Argelia: África
El país Argelia se registró correctamente.

#####
Bienvenido al gestor de información de países
#####
Elija una opción del menú:
1) Agregar un país
2) Actualizar los datos
3) Buscar un país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar Estadísticas
7) Salir
```

Imagen 6: Ejecución de la opción 1: Agregar un país.

```
#####
Bienvenido al gestor de información de países
#####
Elija una opción del menú:
1) Agregar un país
2) Actualizar los datos
3) Buscar un país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar Estadísticas
7) Salir
Ingrese una opción: 2
¿Qué datos desea actualizar?
1) Población
2) Superficie
Ingrese 1 o 2: 1
¿Qué país desea actualizar? uruguay
Ingrese el valor actualizado de la población: 3500000
Los datos de población de Uruguay fueron actualizados correctamente.

#####
Bienvenido al gestor de información de países
#####
Elija una opción del menú:
1) Agregar un país
2) Actualizar los datos
3) Buscar un país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar Estadísticas
7) Salir
```

Imagen 7: Ejecución de la opción 2.1: Actualizar datos de población de un país.

```
#####
Bienvenido al gestor de información de países
#####
Elija una opción del menú:
1) Agregar un país
2) Actualizar los datos
3) Buscar un país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar Estadísticas
7) Salir
Ingrese una opción: 3
¿Qué país desea buscar? arg

Las coincidencias con Arg encontradas son:
Nombre | Población | Superficie [km**2] | Continente
-----
Argentina | 46735004 | 2780400 | América
Argelia | 46810000 | 2381740 | África

#####
Bienvenido al gestor de información de países
#####
Elija una opción del menú:
1) Agregar un país
2) Actualizar los datos
3) Buscar un país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar Estadísticas
7) Salir
Ingrese una opción: █
```

Imagen 8: Ejecución de la opción 3: Buscar país por nombre (se acepta coincidencia parcial o total).

4. Dificultades y conclusiones

4.1. Dificultades a la hora de resolver el programa

Una de las dificultades que surgió durante la realización de este trabajo estuvo relacionada con la actualización del archivo en aquellos casos en que se modifican campos específicos dentro de una línea determinada (por ejemplo, cuando el usuario actualiza la población de un país en particular).

Si bien la lectura del archivo mediante el método DictReader() permitía modificar de manera sencilla el diccionario correspondiente al país solicitado por el usuario, la dificultad residía en la actualización del archivo sin que dicha línea apareciese duplicada al final de este, conteniendo los datos actualizados.

La solución adoptada consistió en generar una lista auxiliar que almacenara, en el orden correspondiente, la totalidad de los diccionarios representativos de cada línea del archivo. De este modo, al alcanzar el diccionario que el usuario deseaba modificar, este se incorporaba a la lista respetando dicho orden, pero incluyendo los valores actualizados.

Una vez completado este procedimiento, la lista auxiliar se emplea para sobrescribir el archivo original, de manera que las líneas quedan almacenadas en el orden correcto, incorporando los cambios correspondientes.

Otra dificultad que surgió durante el desarrollo fue la necesidad de ordenar los países según distintos criterios como nombre, población o superficie, conservando al mismo tiempo todos los datos de cada país, para poder mostrar la información ordenada y completa por pantalla.

Si bien la función **sorted()** era conocida, inicialmente no funcionaba para ordenar listas de diccionarios de la forma planificada. Luego de consultar la documentación oficial, se descubrió el parámetro **key=** que permite definir el criterio de ordenamiento, en conjunto con la función **lambda** que permitía indicarle a **sorted()** qué clave del diccionario debía usar como criterio para el ordenamiento.

Así, la solución se desarrolló en una función que recibe el iterador del archivo CSV, el criterio de ordenamiento como cadena de texto y un parámetro con un valor por defecto **ascendente=False**. Por lo tanto, según el criterio recibido se le asigna la **lambda** correspondiente parámetro **key=**, y el valor de ascendente se pasa como parámetro a **reverse=** para controlar si es de menor a mayor o viceversa.

4.2. Conclusión final

En el presente trabajo se desarrolló una aplicación en Python orientada a la gestión de información de países a partir de los datos almacenados en un archivo CSV. El sistema implementa funcionalidades de búsqueda, filtrado, ordenamiento y generación de estadísticas, dando cumplimiento a los objetivos establecidos en la consigna.

A lo largo del desarrollo se aplicaron conceptos fundamentales correspondientes a la asignatura Programación I, entre los que se destacan el empleo de listas y diccionarios, la

modularización mediante funciones, la lectura y escritura de archivos, las estructuras condicionales y repetitivas, y la validación de entradas. Asimismo, se reforzó la importancia de elaborar un código claro y organizado, con el propósito de facilitar su mantenimiento y comprensión.

Finalmente, el desarrollo del trabajo permitió adquirir experiencia en el ámbito del trabajo colaborativo mediante el empleo de Git y GitHub, lo cual favoreció la coordinación entre los integrantes del equipo y la integración de las distintas partes del proyecto en una solución única y funcional.

5. Bibliografía y links

- Python Software Foundation. (2026). The Python Language Reference (version 3.14)
- Sweigart, A. (2020). Automate the Boring Stuff with Python.
- Apuntes de cátedra. Apertura, lectura, escritura, cierre de archivos
- Apuntes de cátedra. Archivos csv (listas y diccionarios)
- Apuntes de cátedra. Manejo de errores con archivos
- Apuntes de cátedra. Manipulación de archivos en Python
- Apuntes de cátedra. Listas
- Apuntes de cátedra. Diccionarios
- **Link del repositorio:** [juanmreyes91/P1_TP_Integrador: Repositorio para el Trabajo Práctico Integrador de la materia Programación I de la UTN.](https://github.com/juanmreyes91/P1_TP_Integrador)
- **Link del video:**
<https://drive.google.com/file/d/1dfH3JNMHQ8Fpku8xrcqnfFGBYqYgTS3n/view?usp=sharing>